



Universidad Autónoma de Baja California

FACULTAD DE CIENCIAS QUÍMICAS E INGENIERÍA



Microprocesadores y Microcontroladores.

Práctica No. 4: Manejo de la Sección de E/S del Microcontrolador ATmega1280/2560.

Alumno:

Joshua Osorio O. – 1293271

Docente:

Jesús A. García L.

Septiembre/2026

Tabla de contenido

I.	Objetivo	3
II.	Descripción de la practica.....	3
	Lista de materiales	3
III.	Desarrollo de la práctica	3
	Teoría	3
	Ensamblador en línea (inline assembly) para GCC	3
	Convención de llamada a rutinas de ensamblador desde C en AVR-GCC	4
	Técnicas de anti-rebote de botones táctiles.....	4
	Display de siete segmentos de ánodo común.....	4
	Diagrama de conexión	4
	Macros y funciones implementadas	6
	SetBitPort(port, bit) y ClrBitPort(port, bit)	7
	void delay(uint16_t mseg).....	7
	void InitPorts(void).....	7
	void Display7Seg(uint8_t data)	7
	Button_State_t check_Btn(volatile uint8_t *pin_register, uint8_t pin_number)	7
	Código fuente	8
	main.c.....	8
	MyRutines.asm.....	12
	Resultados de simulación	13
	Reflexión y análisis crítico	16
	Dificultades técnicas.....	16
	Fracasos, fallas, bugs y errores.....	17
	Cómo se resolvió.....	17
	Conclusión	17
IV.	Bibliografía.....	18

I. Objetivo

Mediante esta práctica el alumno analizará la implementación de ensamblador en línea (inline assembly) invocado desde C, así como la convención de llamada de AVR-GCC para enlazar una rutina escrita íntegramente en ensamblador con código en C. Asimismo, se familiarizará con la configuración y uso de los puertos de entrada/salida del ATmega2560 —incluyendo el manejo de registros de E/S extendida— y con la aplicación de técnicas de anti-rebote (debounce) para la lectura de botones.

II. Descripción de la practica

Investigar la sintaxis de ensamblador en línea (inline assembly) de GCC, en particular para instrucciones AVR de un solo ciclo como SBI y CBI.

Investigar la convención de llamada de AVR-GCC para invocar, desde C, una función implementada en un archivo de ensamblador puro independiente.

Investigar técnicas de anti-rebote (debounce) por software para la lectura de botones mecánicos.

Partiendo del código base del profesor, implementar las funciones SetBitPort(port, bit) y ClrBitPort(port, bit), y las funciones delay(), InitPorts(), Display7Seg() y check_Btn(), de acuerdo con el diagrama de conexión de la práctica (botones en PK0 y PK1, display de siete segmentos en el Puerto L).

Compilar el proyecto con avr-gcc para el ATmega2560 y generar el archivo .hex correspondiente.

Cargar el programa en la tarjeta Elegoo Mega 2560 mediante XLoader y verificar que una pulsación corta del botón incrementa o decrementa el conteo mostrado en el display de 1 en 1, y que una pulsación larga (mayor a 1 s) lo hace de 5 en 5.

Redactar las conclusiones y comentarios de la práctica, y subir el código y el reporte al repositorio del curso.

Lista de materiales

- Computadora personal (PC) con AVR Studio 5.1 y el toolchain WinAVR (avr-gcc, avr-objcopy).
- Tarjeta Elegoo Mega 2560 (compatible con Arduino Mega 2560, ATmega2560 a 16 MHz).
- Display de siete segmentos de ánodo común, conectado al Puerto L, y dos botones táctiles conectados a PK0 (incremento) y PK1 (decremento).
- XLoader, para cargar el archivo .hex generado en la tarjeta.
- Editor de texto (VS Code) y terminal con acceso a git.
- AVR Instruction Set Manual (DS40002198) y datasheet del ATmega2560.

III. Desarrollo de la práctica

Teoría

Ensamblador en línea (inline assembly) para GCC

GCC permite insertar instrucciones de ensamblador directamente dentro de una función de C mediante la sintaxis extendida `__asm__ __volatile__`("plantilla" : salidas : entradas : registros modificados). En AVR-GCC

esto se usa para instrucciones que no tienen equivalente directo en C, como SBI (Set Bit I/O) y CBI (Clear Bit I/O), que ponen en 1 o en 0, en un único ciclo de reloj, un bit de un registro de E/S. El operando "I" en la plantilla obliga al compilador a resolver, en tiempo de compilación, tanto la dirección de E/S (mediante la macro `_SFR_IO_ADDR`) como el número de bit, como constantes inmediatas válidas para SBI/CBI (direcciones de E/S 0x00–0x1F y bits 0–7), ya que estas instrucciones no admiten operandos calculados en tiempo de ejecución.

Convención de llamada a rutinas de ensamblador desde C en AVR-GCC

AVR-GCC sigue una convención de llamada fija para pasar argumentos a una función: un argumento de 16 bits (como `uint16_t`) se recibe en el par de registros r25:r24 (r24 = byte bajo, r25 = byte alto), y el valor de retorno de una función de 8 bits se entrega en r24. Los registros r18–r27 y r30–r31 se consideran de uso libre (call-clobbered) y no requieren guardarse con push/pop antes de usarse dentro de la subrutina; los registros r2–r17 y r28–r29 sí son call-saved y deben preservarse. Gracias a esta convención, una función como `void delay(uint16_t msec)` puede implementarse íntegramente en un archivo .asm independiente, declararse en C con `extern`, y enlazarse en el paso final de compilación sin necesidad de código adicional de interfaz.

Técnicas de anti-rebote de botones táctiles

Un botón mecánico no realiza una transición limpia entre sus estados: al presionarse o soltarse, el contacto "rebota" físicamente durante algunos milisegundos, generando varias transiciones falsas entre 0 y 1 antes de estabilizarse. La técnica de anti-rebote (debounce) por software más simple consiste en, al detectar un cambio de estado, esperar un intervalo fijo (en esta práctica, 20 ms) y volver a leer el pin: si el nuevo estado coincide con el detectado inicialmente, se considera una transición real; si no, se descarta como rebote. La misma verificación se aplica también al soltar el botón, para evitar contar una sola pulsación como varias.

Display de siete segmentos de ánodo común

El display de siete segmentos utilizado en esta práctica es de ánodo común: los ánodos de los ocho LEDs internos (segmentos A-G más el punto decimal DP) están unidos entre sí en un único pin COM, alimentado con 5 V. Cada segmento se enciende de forma individual llevando su propio cátodo a nivel bajo (0 lógico) a través de una resistencia limitadora de corriente; si el cátodo se deja en alto, el LED permanece apagado porque no hay diferencia de potencial entre su ánodo, fijo a 5 V, y su cátodo. Esta es la lógica inversa a la de un display de cátodo común, donde el pin común se conecta a tierra y cada segmento se enciende con un 1 lógico. Por esta razón, en el firmware el Puerto L se inicializa en 0xFF (todos los segmentos apagados) en vez de 0x00, y la tabla de consulta `seg_table[16]` no guarda el patrón "natural" de cada dígito, sino su complemento bit a bit: un bit en 0 enciende el segmento correspondiente y un bit en 1 lo apaga.

Diagrama de conexión

Los botones de incremento y decremento se conectan a PK0 y PK1 respectivamente, configurados como entrada con resistencia de pull-up interna habilitada (en reposo el pin lee 1; al presionar el botón, que lo conecta a tierra, el pin lee 0). El display de siete segmentos, de ánodo común, se conecta íntegramente al Puerto L, configurado como salida.

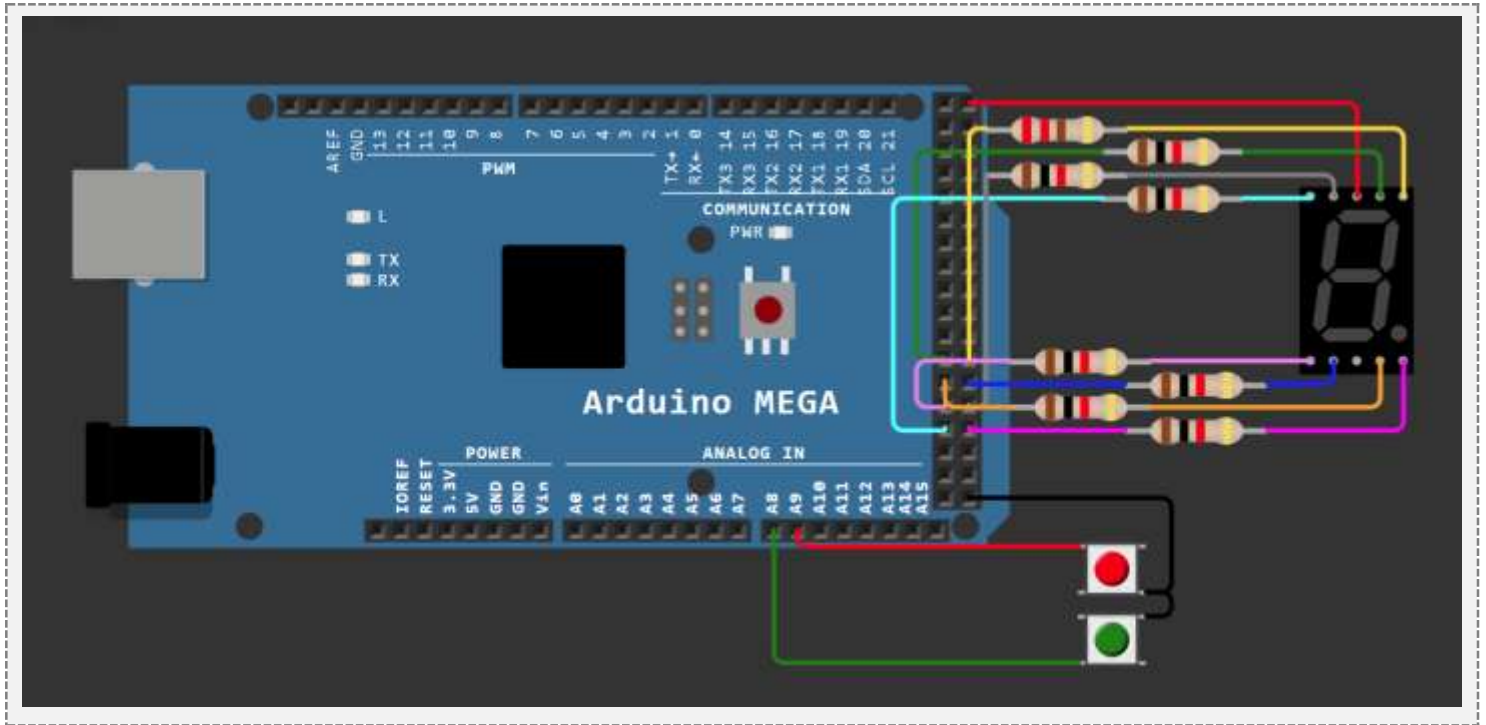


Ilustración 1: Diagrama de conexión del circuito (botones en PK0/PK1, display de 7 segmentos en el Puerto L).

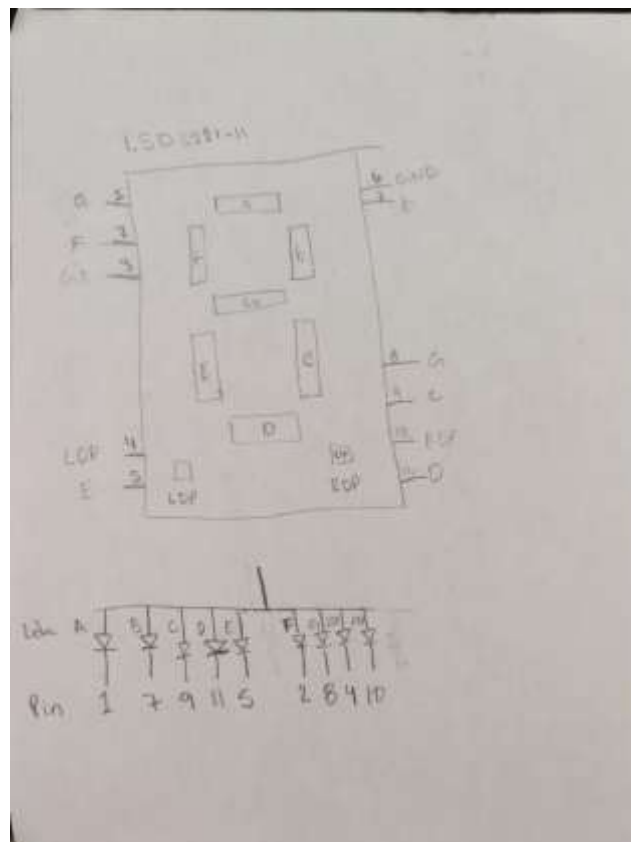


Ilustración 2: Display 7 segmentos LSD 3221-11

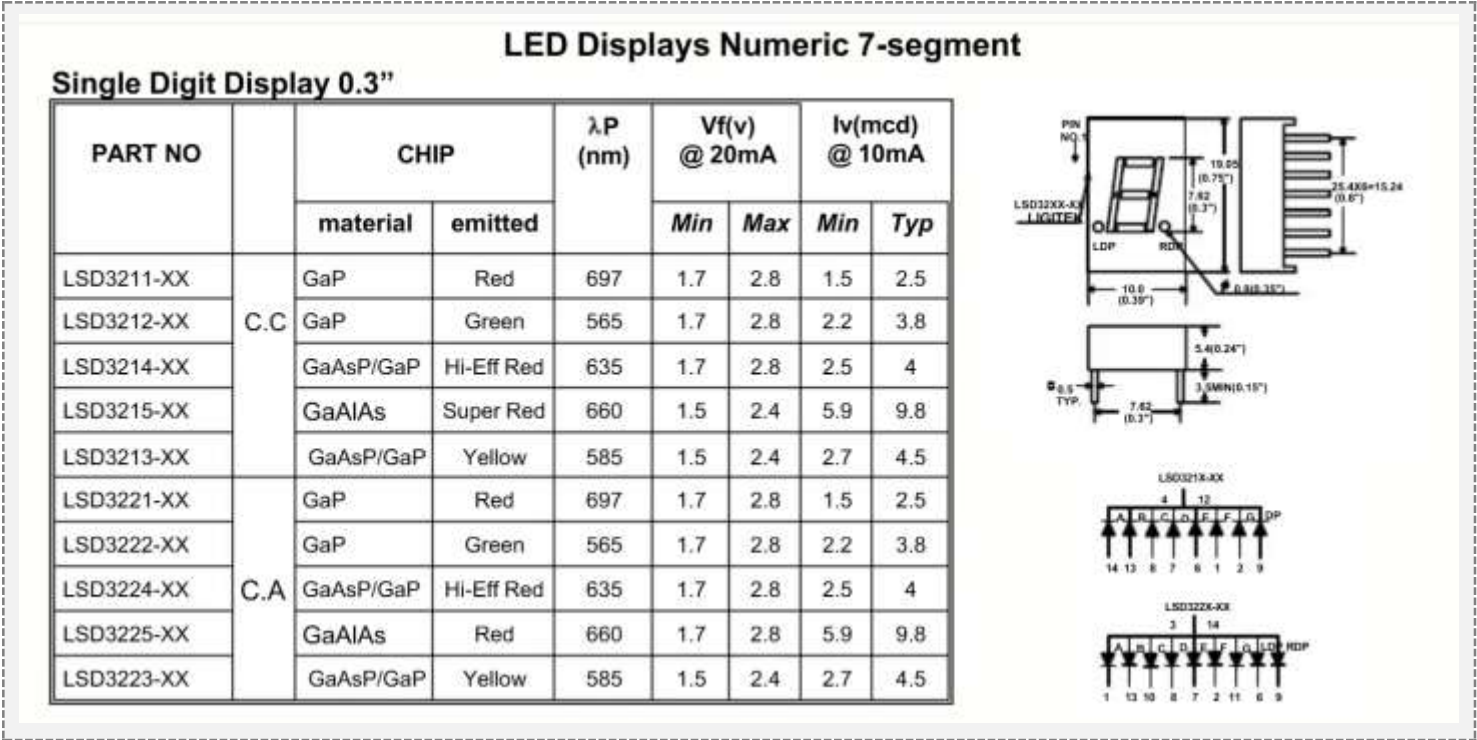


Ilustración 3: LSD3221-XX Datasheet.

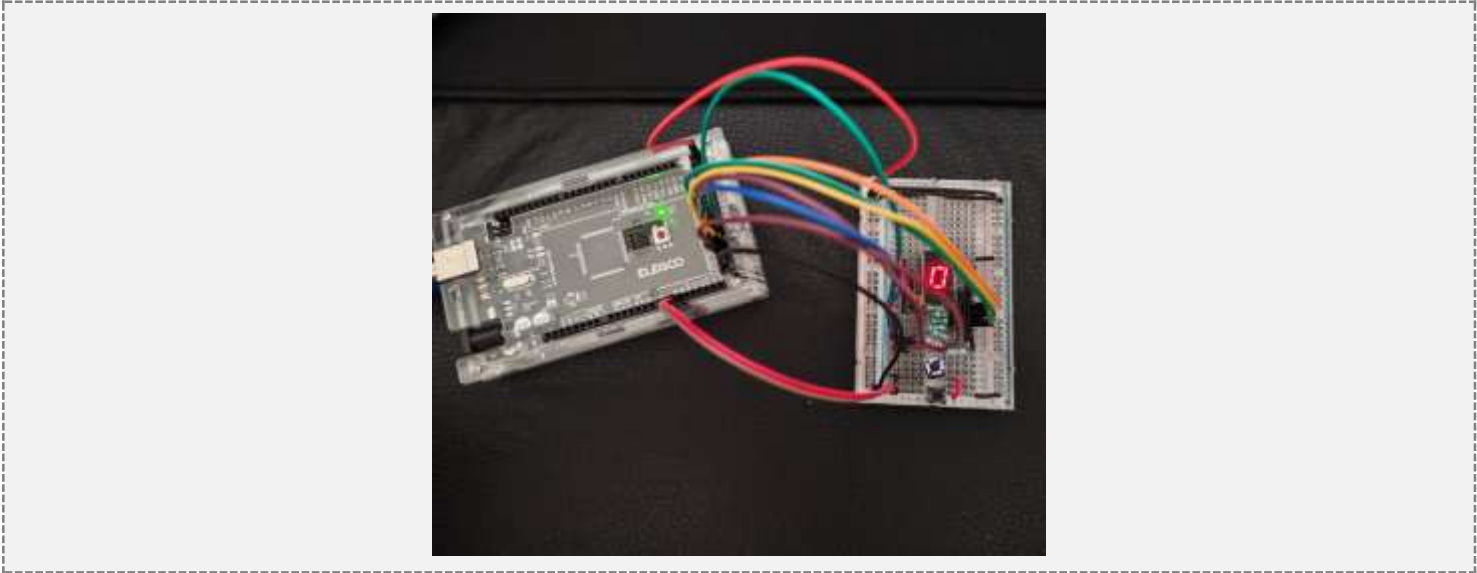


Ilustración 1: Implementación física

Macros y funciones implementadas

SetBitPort(port, bit) y ClrBitPort(port, bit)

Se implementaron como funciones reales de ensamblador en el archivo MyRutines.asm (no como macros de ensamblador en línea): reciben un puntero al registro de E/S y el número de bit, leen el registro con la instrucción `ld` a través del registro `Z`, arman una máscara de un bit mediante un lazo de corrimiento (`lsl`) controlado por el número de bit recibido, y escriben el resultado de vuelta con `st`. Al direccionar el registro como una posición de memoria en vez de usar una instrucción de E/S de un solo ciclo (`SBI/CBI`), esta implementación no está limitada al rango de E/S clásica (`0x00-0x1F`): alcanza sin problema los registros de E/S extendida del ATmega2560, como `DDRK/PORTK` y `DDRL/PORTL` (dirección de memoria $\geq 0x100$), que es justo lo que exige el diagrama de esta práctica. Como contraparte, al no ser instrucciones de un solo ciclo, son más lentas que `SBI/CBI`, aunque a cambio sí aceptan un número de bit calculado en tiempo de ejecución, algo que `SBI/CBI` no permite porque su operando de bit debe ser una constante conocida en tiempo de compilación.

void delay(uint16_t msec)

Implementada íntegramente en ensamblador puro en el archivo MyRutines.asm y enlazada con `main.c` en el paso final de compilación. Se calibró, contando ciclos de instrucción a partir de la tabla del AVR Instruction Set Manual, para que cada iteración del lazo de milisegundos consuma exactamente 16 000 ciclos de reloj; a $F_{CPU} = 16 \text{ MHz}$ esto equivale a 1.000000 ms por milisegundo solicitado, dentro de la exactitud de $\pm 5 \mu s$ pedida en la práctica.

void InitPorts(void)

Recorre los ocho bits del Puerto L en un ciclo, llamando a `SetBitPort()` dos veces por iteración: una para poner en 1 el bit correspondiente de `DDRL` (configurándolo como salida) y otra para ponerlo en 1 en `PORTL`, dejando el display completamente apagado al inicio (`0xFF`); al ser de ánodo común, un segmento apagado corresponde a un 1 lógico en su pin. Después configura los pines `PK0` y `PK1` como entrada llamando a `ClrBitPort()` sobre `DDRK`, y habilita su resistencia de pull-up interna llamando a `SetBitPort()` sobre `PORTK`.

void Display7Seg(uint8_t data)

Traduce los 4 bits menos significativos de `data` a su patrón de segmentos correspondiente mediante una tabla de consulta (`seg_table[16]`) y lo escribe directamente en el Puerto L. Al tratarse de un display de ánodo común, cada segmento se enciende con el bit en bajo, por lo que la tabla almacena el patrón ya invertido para cada dígito hexadecimal (`0-F`).

Button_State_t check_Btn(volatile uint8_t *pin_register, uint8_t pin_number)

Determina si el botón indicado está libre, fue presionado brevemente o fue mantenido presionado más de un segundo, aplicando anti-rebote tanto al detectar el flanco de bajada (al presionar) como el de subida (al soltar). Devuelve un valor del tipo enumerado `Button_State_t`: `eBtnUndefined` (0) si no fue presionado, `eBtnShortPressed` (1) si la pulsación fue corta y `eBtnLongPressed` (5) si fue larga (umbral $> 1000 \text{ ms}$); este valor se suma o resta directamente al contador en el ciclo principal, por lo que una pulsación larga incrementa o decrementa el conteo de 5 en 5.

Código fuente

El programa se organizó en dos archivos fuente que se compilan y enlazan juntos compilar.sh: main.c, con la lógica en C, y MyRutines.asm, con la rutina delay() en ensamblador puro.

main.c

```

/*
 * U.A.B.C. - Facultad de Ciencias Químicas e Ingeniería
 * 36290 Microprocesadores y Microcontroladores
 * Practica 4 - Manejo de la seccion de E/S del ATmega1280/2560
 * Alumno: Joshua Osorio O. - 1293271
 *
 * Descripción:
 *   Revisa el estado de los botones conectados en
 *   PK0 (incremento)
 *   PK1 (decremento)
 *   Segun la duracion de la pulsacion
 *   (corta o larga, umbral > 1 s) actualiza un contador que se despliega en el
 *   display de 7 segmentos conectado al Puerto L.
 *
 * Notas de plataforma:
 *   - Los botones se configuran con PULL-UP interno (DDRx=0, PORTx=1):
 *     en reposo el pin lee 1, al presionar el boton se conecta a GND
 *     y el pin lee 0.
 *   - SetBitPort/ClrBitPort estan implementadas como funciones reales en
 *     ensamblador (MyRutines.asm), que acceden al registro con ld/st a
 *     traves de un puntero (registro Z). A diferencia de una macro basada
 *     en SBI/CBI (limitada a E/S 0x00-0x1F), estas funciones si alcanzan
 *     PORTK y PORTL, que en el ATmega2560 son E/S EXTENDIDA (direccion de
 *     memoria >= 0x100).
 *   - delay() tambien esta implementada en ensamblador puro en el archivo
 *     externo MyRutines.asm, calibrada a F_CPU = 16 MHz. main.c y
 *     MyRutines.asm se compilan y enlazan juntos (ver src/compilar.sh).
 *   - Igual que en el codigo del profesor, check_Btn() no solo indica
 *     si hubo pulsacion: su valor de retorno (0/1/5) se suma o resta
 *     directamente al contador ("count += check_Btn(...)", por lo
 *     que una pulsacion larga incrementa/decrementa de 5 en 5.
 */

#include <avr/io.h>

/* Asignacion de pines segun el diagrama */
#define INC_BUTTON PK0 /* Boton de incremento -> PK0 */
#define DEC_BUTTON PK1 /* Boton de decremento -> PK1 */
#define BUTTON_DDR DDRK
#define BUTTON_PORT PORTK

```

```

#define BUTTON_PIN PINK

#define DISPLAY_DDR DDRL
#define DISPLAY_PORT PORTL /* Display de 7 segmentos -> Puerto L */

/* Parametros de tiempo */
#define DEBOUNCE_MS 20U /* Ventana de anti-rebote */
#define LONG_PRESS_MS 1000U /* Umbral de pulsacion larga: > 1 seg */

typedef enum ButtonStates
{
    eBtnUndefined = 0,
    eBtnShortPressed = 1,
    eBtnLongPressed = 5
} Button_State_t;

/* Prototipos del 3 al 6 */
extern void delay(uint16_t mseg); /* Implementada en
MyRutines.asm */
extern void SetBitPort(volatile uint8_t *port, uint8_t bit); /* Implementada en
MyRutines.asm */
extern void ClrBitPort(volatile uint8_t *port, uint8_t bit); /* Implementada en
MyRutines.asm */
void InitPorts(void);
void Display7Seg(uint8_t data);
Button_State_t check_Btn(volatile uint8_t *pin_register, uint8_t pin_number);

/*
 * Tabla de segmentos para digitos 0-F.
 * Bit7=A Bit6=B Bit5=C Bit4=D Bit3=E Bit2=F Bit1=G Bit0=DP
 * (orden fisico real:
 * D42=A,
 * D43=B,
 * D44=C,
 * D45=D,
 * D46=E,
 * D47=F,
 * D48=G,
 * D49=DP
 * -- ver diagram.json.
 * Es el orden "al derecho": el pin mas bajo (D42=PL7) a la letra A,
 * subiendo hasta D49=PL0=DP.
 * Activo en BAJO (display comun anodo, COM -> 5V): un segmento se
 * enciende con 0.
 */
static const uint8_t seg_table[16] =

```

```

{
    0x03, /* 0 */
    0x9F, /* 1 */
    0x25, /* 2 */
    0x0D, /* 3 */
    0x99, /* 4 */
    0x49, /* 5 */
    0x41, /* 6 */
    0x1F, /* 7 */
    0x01, /* 8 */
    0x09, /* 9 */
    0x11, /* A */
    0xC1, /* b */
    0x63, /* C */
    0x85, /* d */
    0x61, /* E */
    0x71 /* F */
};

int main(void)
{
    uint8_t count = 0;

    UCSR0B &= (uint8_t)~(1 << TXEN0); /* Deshabilita TX de UART0 */

    InitPorts();

    while (1)
    {
        /* El valor de retorno (0/1/5) se suma/resta directamente:
         * sin pulsar = +0, pulsacion corta = +-1, pulsacion larga = +-5 */
        count += check_Btn(&BUTTON_PIN, INC_BUTTON);
        count -= check_Btn(&BUTTON_PIN, DEC_BUTTON);

        Display7Seg(count & 0x0F);
    }
}

/* 4 Inicializacion de los puertos utilizados segun el diagrama */
void InitPorts(void)
{
    uint8_t i;

    /* Display de 7 segmentos en PORTL como salida, inicia apagado
     * (comun anodo: 1 = segmento apagado). Se arma bit a bit con
     * SetBitPort porque PORTL es E/S extendida. */

```

```

for (i = 0; i < 8; i++)
{
    SetBitPort(&DISPLAY_DDR, i);
    SetBitPort(&DISPLAY_PORT, i);
}

/* Botones en PK0/PK1 como entrada con PULL-UP interno */
ClrBitPort(&BUTTON_DDR, INC_BUTTON); /* Definir como entrada */
ClrBitPort(&BUTTON_DDR, DEC_BUTTON);
SetBitPort(&BUTTON_PORT, INC_BUTTON); /* Habilita pull-up */
SetBitPort(&BUTTON_PORT, DEC_BUTTON);
}

/* (5) Despliega "data" (4 bits) sobre el display de 7 segmentos */
void Display7Seg(uint8_t data)
{
    DISPLAY_PORT = seg_table[data & 0x0F]; /* aseguro que los bits altos no tengan
basura*/
}

/*
 * 6 Estado del boton en pin_register:pin_number, distinguiendo
 * pulsacion corta / larga (> 1 s), con anti-rebote a la entrada y a
 * la salida de la pulsacion.
 */
Button_State_t check_Btn(volatile uint8_t *pin_register, uint8_t pin_number)
{
    uint16_t held_ms = 0;

    /* Reposo (pull-up): nivel alto = boton libre */
    if ((*pin_register & (1 << pin_number)) != 0)
    {
        return eBtnUndefined;
    }
    /* Anti-rebote de flanco de bajada: confirma que el nivel bajo persiste */
    delay(DEBOUNCE_MS);
    if ((*pin_register & (1 << pin_number)) != 0)
    {
        return eBtnUndefined; /* fue rebote, no una pulsacion real */
    }

    /* Mide cuanto tiempo permanece presionado */
    while ((*pin_register & (1 << pin_number)) == 0)
    {
        delay(1);
        held_ms++;
    }
}

```

```

}
/* Anti-rebote de flanco de subida (al soltar) */
delay(DEBOUNCE_MS);

return (held_ms > LONG_PRESS_MS) ? eBtnLongPressed : eBtnShortPressed;
}

```

MyRutines.asm

```

#include <avr/io.h>

.section .text
.global delay
.global SetBitPort
.global ClrBitPort

delay:
    sbiw    r24, 0
    breq    delay_fin

delay_lazo_ms:
    ldi     r30, 0x9E
    ldi     r31, 0x0F

delay_lazo_us:
    sbiw    r30, 1
    brne    delay_lazo_us

    nop
    nop
    nop

    sbiw    r24, 1          ; mseg--
    brne    delay_lazo_ms

delay_fin:
    ret

SetBitPort:
    movw    r30, r24        ; Z = port
    ld      r18, Z          ; r18 = *port
    ldi     r19, 1

SetBitPort_mask:
    cpi     r22, 0
    breq    SetBitPort_apply
    lsl     r19
    dec     r22

```

```

    rjmp    SetBitPort_mask
SetBitPort_apply:
    or      r18, r19
    st      Z, r18
    ret

ClrBitPort:
    movw    r30, r24        ; Z = port
    ld      r18, Z          ; r18 = *port
    ldi     r19, 1
ClrBitPort_mask:
    cpi     r22, 0
    breq    ClrBitPort_apply
    lsl     r19
    dec     r22
    rjmp    ClrBitPort_mask
ClrBitPort_apply:
    com     r19              ; ~(1<<bit)
    and     r18, r19
    st      Z, r18
    ret

```

Resultados de simulación

Se verificó el funcionamiento cargando el programa compilado (main.hex) en la tarjeta Elegoo Mega 2560 mediante XLoader, confirmando que una pulsación corta del botón en PK0 incrementa el conteo mostrado en el display de siete segmentos de 1 en 1, una pulsación corta en PK1 lo decreuenta de 1 en 1, y que mantener presionado cualquiera de los dos botones por más de un segundo hace que el conteo cambie de 5 en 5.

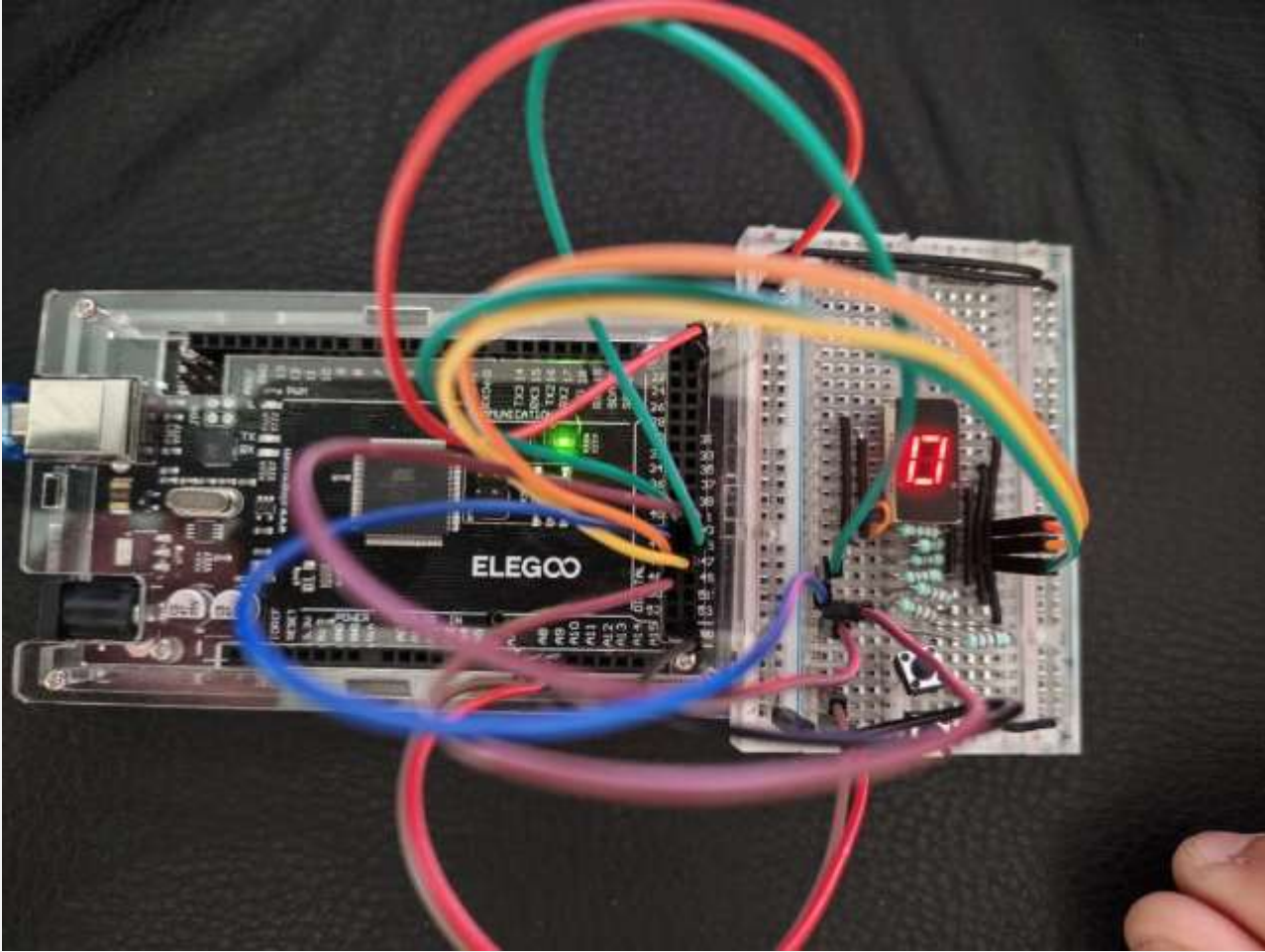


Ilustración 2: Display mostrando el conteo inicial (0).

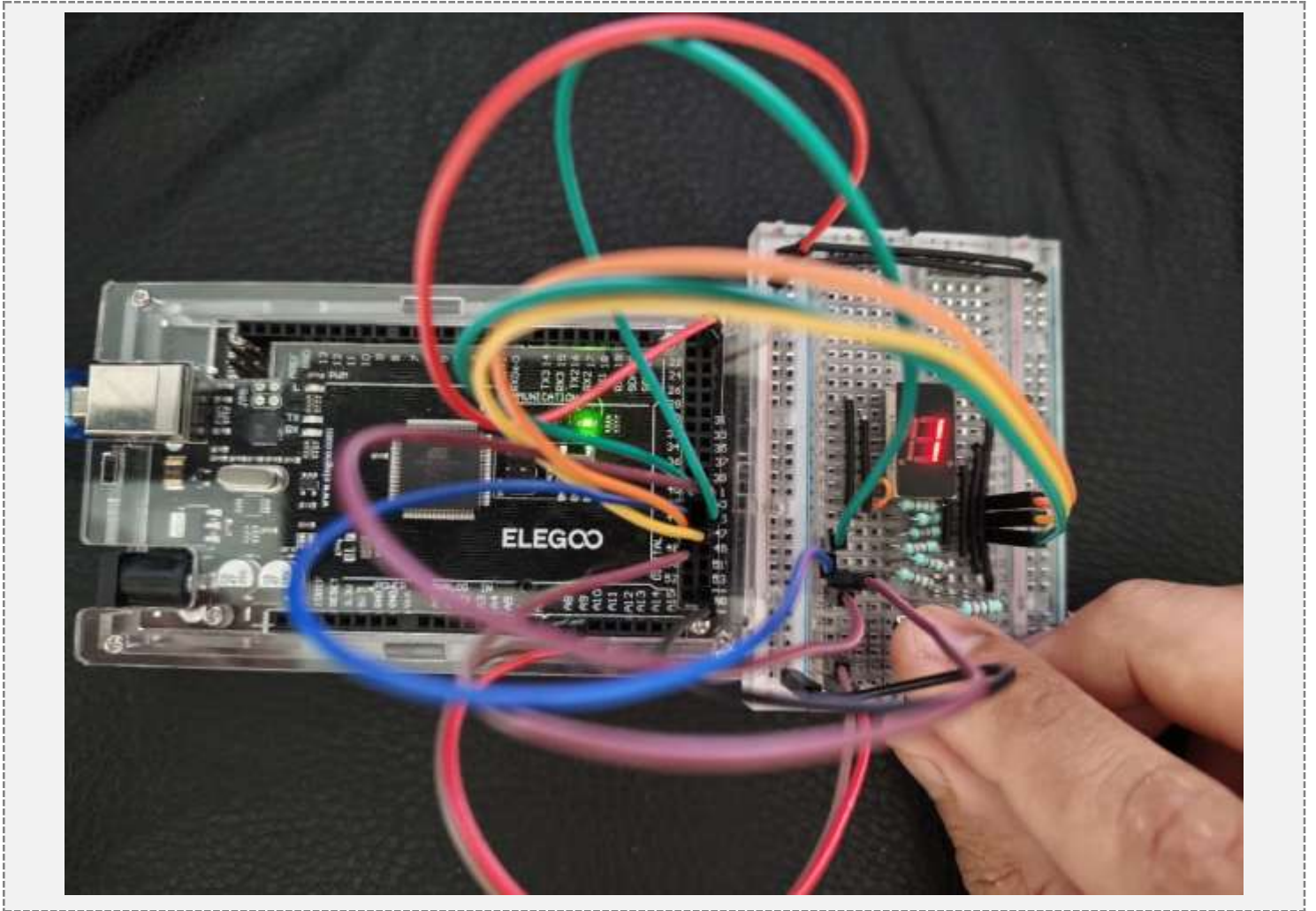


Ilustración 3: Pulsación corta: incremento/decremento de 1 en 1.

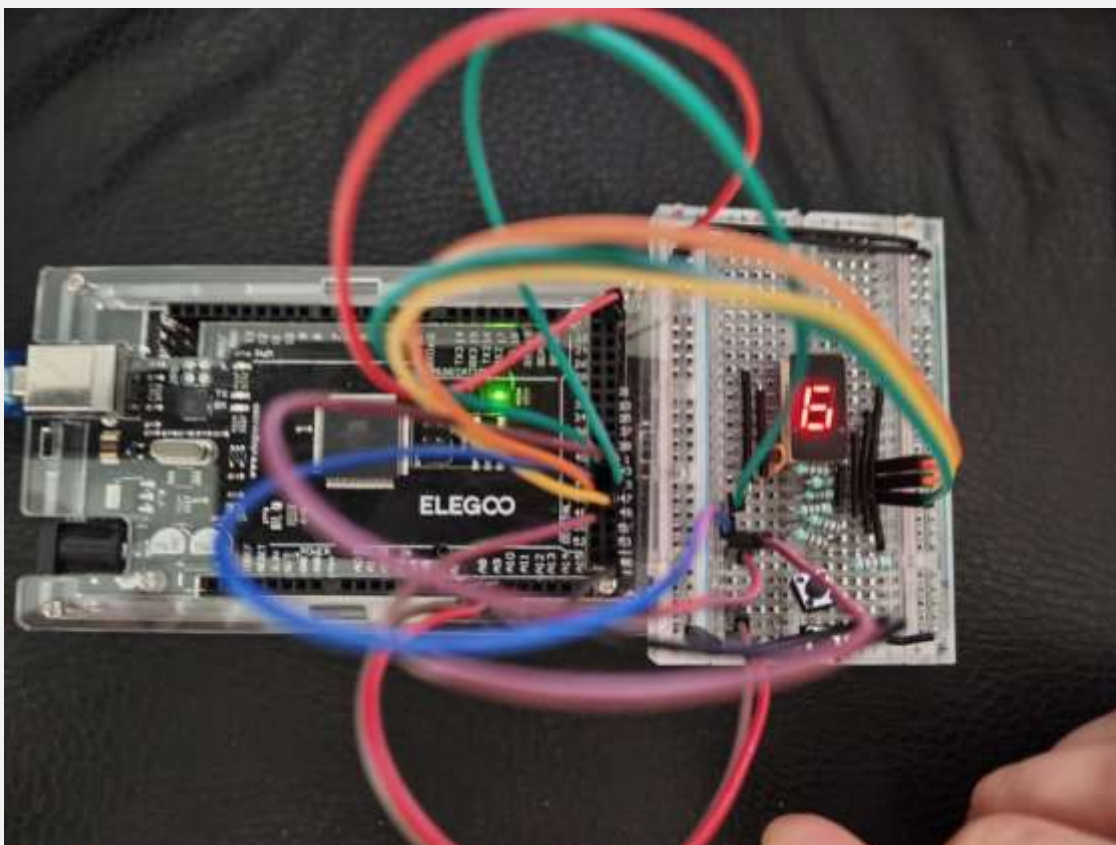


Ilustración 4: Pulsación larga (> 1 s): incremento/decremento de 5 en 5.

Reflexión y análisis crítico

Dificultades técnicas

La principal dificultad fue que la primera versión de SetBitPort/ClrBitPort, basada en las instrucciones SBI/CBI de un solo ciclo, no podía usarse en los puertos que exige el diagrama de esta práctica: en el ATmega2560, a diferencia de microcontroladores más pequeños de la familia AVR, el Puerto K y el Puerto L son registros de E/S extendida y quedan fuera del rango de direcciones que admite el operando inmediato de SBI/CBI. Otra dificultad fue calibrar la rutina delay() en ensamblador para que cumpliera la exactitud de $\pm 5 \mu s$ pedida, ya que cualquier instrucción de relleno mal contada desvía el resultado de los 16 000 ciclos por milisegundo objetivo.

Otra dificultad fue que el display de siete segmentos disponible era una pieza antigua sin datasheet claro: no venía especificado en ningún lado qué pin físico correspondía a cada segmento (A-G) ni al punto decimal, por lo que fue necesario determinar esa asignación de forma experimental, alimentando cada pin de manera individual y observando qué segmento encendía, para poder construir el mapeo pin-a-letra. A esto se sumó tener que compilar y cargar el programa a la tarjeta en un solo paso: se amplió `compile.sh` para que, además de generar el `.hex` a partir de `main.c` y `MyRoutines.asm`, invoque directamente a `avrdude` y suba el binario a la Elegoo Mega 2560, en vez de depender de un programa externo (XLoader) para ese último paso.

Fracasos, fallas, bugs y errores

El primer intento de `InitPorts()` usó las macros `SetBitPort/ClrBitPort` basadas en SBI/CBI también para los pines de `PORTK` y `PORTL`, lo que provocó un error de compilación ("operand out of range"), ya que el ensamblador rechaza en tiempo de compilación cualquier dirección de $E/S \geq 0x20$ en el operando inmediato de SBI/CBI. En una prueba temprana de `check_Btn()`, aplicar el anti-rebote solo al presionar (sin verificarlo también al soltar) hacía que, ocasionalmente, una sola pulsación se contara dos veces por el rebote mecánico al retirar el dedo del botón.

Al integrar la subida automática a la tarjeta en `compile.sh`, `avrdude` devolvió el error "Can't find programmer id 'wiring'": el toolchain WinAVR instalado es de 2010 y su `avrdude.conf` no incluye ese identificador de programador, agregado en versiones posteriores de `avrdude` específicamente para el bootloader de las tarjetas Arduino/Elegoo Mega. Por otro lado, al determinar el mapeo de pines del display de siete segmentos antiguo, la primera asignación usada (basada en una suposición del orden de los pines, no en una verificación real) resultó incorrecta: en la tarjeta se mostraba un dígito distinto al esperado (aparecía un "6" en vez del dígito correcto), lo que evidenció que el orden real de los segmentos estaba invertido respecto al asumido en la tabla `seg_table` del firmware.

Cómo se resolvió

Se reescribieron `SetBitPort` y `ClrBitPort` como funciones reales de ensamblador en vez de macros basadas en SBI/CBI: reciben un puntero al registro y el número de bit, acceden al registro con las instrucciones `ld/st` a través del registro `Z` y arman la máscara de bit con un lazo de corrimiento en tiempo de ejecución. Al direccionar el registro por memoria en vez de por E/S , esta versión sí alcanza los registros de E/S extendida (`PORTK` y `PORTL`), por lo que `InitPorts()` pudo terminar usando `SetBitPort()/ClrBitPort()` de forma consistente para los dos puertos de la práctica. El doble conteo por rebote se corrigió agregando una segunda espera de anti-rebote inmediatamente después de detectar que el botón fue soltado, antes de aceptar la pulsación como válida.

El error de `avrdude` se resolvió cambiando el identificador de programador de "wiring" a "arduino" en la línea agregada a `compile.sh`; ambos hablan el mismo protocolo (STK500v1) a 115200 baudios, que es el que usa el bootloader de fábrica de la Mega2560, por lo que "arduino" funciona igual de bien y sí está definido en el `avrdude.conf` de ese WinAVR. El problema del display se resolvió verificando uno por uno, físicamente, qué pin encendía cada segmento, confirmando que la conexión real ($D42 \rightarrow A \dots D48 \rightarrow G, D49 \rightarrow DP$) era la inversa de la que se había asumido originalmente; con ese orden real confirmado, se recalculó la tabla `seg_table` invirtiendo bit a bit cada uno de sus 16 valores, además de mantener la lógica activa en bajo que exige el ánodo común (un segmento se enciende con 0, no con 1), con lo que el display volvió a mostrar el dígito correcto.

Conclusión

Esta práctica permitió comprobar los límites de las instrucciones de un solo ciclo del AVR (SBI/CBI), que solo aplican a la E/S clásica y no a los puertos añadidos en microcontroladores más grandes como el ATmega2560, y contrastarlas con el acceso por memoria (`ld/st` a través de un puntero), que sí alcanza cualquier registro mapeado en memoria —incluida la E/S extendida— a cambio de más ciclos de ejecución.

También dejó claro que trabajar con un display de ánodo común invierte la lógica habitual de "1 enciende": hay que inicializar los puertos en alto y usar una tabla de segmentos complementada para que cada patrón se muestre correctamente. Igualmente reforzó la importancia de tratar el anti-rebote como un problema de tiempo —esperar y volver a verificar— en ambos flancos de una pulsación, y no solo al presionar, así como la convención de llamada de AVR-GCC como el mecanismo que permite combinar, sin código de interfaz adicional, una rutina crítica en tiempo (delay, en ensamblador puro) con la lógica de control de más alto nivel escrita en C.

IV. Bibliografía

- Microchip Technology / Atmel. (2021). AVR® Instruction Set Manual, DS40002198.
- Microchip Technology / Atmel. ATmega640/V-1280/V-1281/V-2560/V-2561/V Datasheet, DS40002211A.
- Universidad Autónoma de Baja California, Facultad de Ciencias Químicas e Ingeniería. (2026). Práctica 4: Manejo de la sección de E/S del microcontrolador ATmega1280/2560.
- Free Software Foundation. Using the GNU Compiler Collection (GCC) — How to Use Inline Assembly Language in C Code (Extended Asm).